```
from PySide6.QtWidgets import (QDialog, QHBoxLayout, QLineEdit, QPushButton,
                               QLabel, QCheckBox, QFrame)
from PySide6.QtCore import Qt, Signal, QSize
from PySide6.QtGui import QPainter, QPen, QColor, QCursor

class CustomVectorButton(QPushButton):
    def __init__(self, arrow_type=None, is_close=False, parent=None):
        super().__init__(parent)
        self.arrow_type = arrow_type
        self.is_close = is_close
        self.setFixedSize(QSize(24, 24))
        self.setFlat(True)
        # Ensure buttons use standard pointer cursors
        self.setCursor(QCursor(Qt.CursorShape.PointingHandCursor))

    def paintEvent(self, event):
        super().paintEvent(event)
        with QPainter(self) as painter:
            painter.setRenderHint(QPainter.RenderHint.Antialiasing)
            pen = QPen(QColor("#444444"), 2)
            pen.setCapStyle(Qt.PenCapStyle.RoundCap)
            painter.setPen(pen)

            rect = self.rect()
            center_point = rect.center()
            cx, cy = center_point.x(), center_point.y()

            if self.is_close:
                offset = 5
                painter.drawLine(cx - offset, cy - offset, cx + offset, cy + offset)
                painter.drawLine(cx + offset, cy - offset, cx - offset, cy + offset)
            elif self.arrow_type == "up":
                painter.drawLine(cx - 5, cy + 2, cx, cy - 3)
                painter.drawLine(cx, cy - 3, cx + 5, cy + 2)
            elif self.arrow_type == "down":
                painter.drawLine(cx - 5, cy - 3, cx, cy + 2)
                painter.drawLine(cx, cy + 2, cx + 5, cy - 3)

class TabFindDialog(QDialog):
    find_requested = Signal(str, bool, bool)
    closed = Signal()

    def __init__(self, parent=None):
        super().__init__(parent, Qt.WindowFlags.FramelessWindowHint | Qt.WindowFlags.SubWindow)
        self.setObjectName("FindDialog")

        # FIXED: Forces the cursor profile to an arrow to bypass the viewport's inherited I-beam profile.
        self.setCursor(QCursor(Qt.CursorShape.ArrowCursor))

        self.init_ui()

    def init_ui(self):
        self.setMinimumWidth(440)
        self.setFixedHeight(36)

        layout = QHBoxLayout(self)
        layout.setContentsMargins(8, 0, 8, 0)
        layout.setSpacing(8)
        layout.setAlignment(Qt.AlignmentFlag.AlignVCenter)

        # FIXED: The checkmark SVG string is percent-encoded (%23 replacing #, %3E replacing >, etc.)
        # This guarantees it renders properly across all platforms.
        self.setStyleSheet("""
            QDialog#FindDialog {
                background-color: #f6f6f6;
                border: 1px solid #ababab;
                border-radius: 6px;
            }
            QLineEdit {
                border: 1px solid #b0b0b0;
                border-radius: 3px;
                padding: 3px 6px;
                background: #ffffff;
                color: #222222;
                font-size: 12px;
            }
            QLineEdit:focus { border: 1px solid #1473e6; }
            QLabel { color: #4b4b4b; font-size: 12px; background: transparent; }

            QCheckBox { color: #222222; font-size: 12px; spacing: 4px; }
            QCheckBox::indicator {
                width: 14px;
                height: 14px;
                border: 1px solid #b0b0b0;
                border-radius: 2px;
                background-color: #ffffff;
```

```
            }
            QCheckBox::indicator:hover { border: 1px solid #1473e6; }
            QCheckBox::indicator:checked {
                background-color: #1473e6;
                border: 1px solid #1473e6;
                image: url("data:image/svg+xml,%3Csvg xmlns='w3.org' viewBox='0 0 24 24' fill='none' stroke='white' stroke-width='4'
stroke-linecap='round' stroke-linejoin='round'%3E%3Cpolyline points='20 6 9 17 4 12'%3E%3C/polyline%3E%3C/svg%3E");
            }
            QPushButton { background-color: transparent; border: none; border-radius: 3px; }
            QPushButton:hover { background-color: #e1e1e1; }
        """)

        self.search_input = QLineEdit(self)
        self.search_input.setPlaceholderText("Find text...")
        self.search_input.setMinimumWidth(160)
        self.search_input.setFixedHeight(24)
        self.search_input.textChanged.connect(self.on_text_changed)
        layout.addWidget(self.search_input)

        self.counter_label = QLabel("0 of 0", self)
        self.counter_label.setMinimumWidth(55)
        self.counter_label.setAlignment(Qt.AlignmentFlag.AlignRight | Qt.AlignmentFlag.AlignVCenter)
        layout.addWidget(self.counter_label)

        line = QFrame(self)
        line.setFrameShape(QFrame.Shape.VLine)
        line.setFrameShadow(QFrame.Shadow.Plain)
        line.setStyleSheet("color: #d0d0d0;")
        line.setFixedHeight(18)
        layout.addWidget(line)

        self.prev_btn = CustomVectorButton(arrow_type="up", parent=self)
        self.prev_btn.clicked.connect(self.find_prev)
        layout.addWidget(self.prev_btn)

        self.next_btn = CustomVectorButton(arrow_type="down", parent=self)
        self.next_btn.clicked.connect(self.find_next)
        layout.addWidget(self.next_btn)

        self.case_check = QCheckBox("Match Case", self)
        self.case_check.setCursor(QCursor(Qt.CursorShape.PointingHandCursor))
        self.case_check.setFixedHeight(24)
        self.case_check.stateChanged.connect(self.on_text_changed)
        layout.addWidget(self.case_check)

        self.close_btn = CustomVectorButton(is_close=True, parent=self)
        self.close_btn.clicked.connect(self.close)
        layout.addWidget(self.close_btn)

    def update_counter(self, current: int, total: int):
        self.counter_label.setText(f"{current} of {total}")

    def on_text_changed(self):
        text = self.search_input.text()
        if text:
            self.find_requested.emit(text, True, self.case_check.isChecked())
        else:
            self.update_counter(0, 0)

    def find_next(self):
        if self.search_input.text():
            self.find_requested.emit(self.search_input.text(), True, self.case_check.isChecked())

    def find_prev(self):
        if self.search_input.text():
            self.find_requested.emit(self.search_input.text(), False, self.case_check.isChecked())

    def keyPressEvent(self, event):
        if event.key() in (Qt.Key.Key_Return, Qt.Key.Key_Enter):
            if event.modifiers() & Qt.KeyboardModifier.ShiftModifier:
                self.find_prev()
            else:
                self.find_next()
            event.accept()
        elif event.key() == Qt.Key.Key_Escape:
            self.close()
        else:
            super().keyPressEvent(event)

    def closeEvent(self, event):
        self.closed.emit()
        super().closeEvent(event)
```